

END-OF-COURSE PROJECT · MILESTONE 1 · COMPANION TO THE PROBLEM STATEMENT TEMPLATE

Writing your proposal — what to watch out for

Ajay Vikram Singh · Course Designer and Instructor, PE6201

Please read this first. These are **suggestions for building your proposal**, which is due **Sunday 23 August 2026, 23:59 SGT**. The Problem Statement itself is **not graded** — but it is **mandatory**: submitting it on time is a requirement for your End-of-Course Project to be counted complete. It is formative, so use it to change direction while changing direction is still cheap. Read it alongside the Project Rubric.

Every pattern below appeared in the statements submitted so far. The test for each section is the same: **does it get you closer to a working system?**

HOW THESE SECTIONS MAP TO THE PROJECT MARK

Template section	Project Rubric criterion	ILO
1 · Working title	4 · Demonstration & Communication (25%)	4
2 · The problem, and why it matters	1 · Problem Statement & Significance (15%)	4
3 · Who it is for, persona and domain	1 · Problem Statement & Significance (15%)	4
4 · Why AI — and which kind	2 · Business & Technical Trade-offs (25%)	1, 3
5 · Proposed approach — build vs buy	2 · Business & Technical Trade-offs (25%)	2, 3
6 · Data	3 · Implementation — code & repository (35%)	2
7 · Success metric and evaluation	3 · Implementation — code & repository (35%)	2, 3
8 · Risks, limitations, responsible use	2 · Trade-offs (25%) · 3 · Implementation (35%) · 4 · Demonstration (25%)	2, 3, 4
9 · Smallest first version	3 · Implementation — code & repository (35%)	2

Read this the right way. This milestone is formative, and it seeds criteria 1 and 2 directly — that is what the template says and it still holds. The rest of the table shows where each answer *eventually* lands when the full project is marked. Note what that means: **criterion 3 is the largest single block of the project mark at 35%, and it is decided by sections 6, 7 and 9** — your data, your evaluation, and your first working slice. That work starts now, not in Week 6.

Section 8 spans three criteria — and it is the section most statements give two lines to. Risk is a named business axis in criterion 2; the guardrails you actually build are implementation under criterion 3, because a mitigation you only describe is not a mitigation; and criterion 4 asks your demonstration to articulate limitations out loud. Budget more space for it than you think you need.

The four Intended Learning Outcomes (ILOs), in short. **1** — distinguish AI technologies and select the right one for the problem class. **2** — build and validate an end-to-end AI solution. **3** — evaluate design alternatives on business and technical factors, and justify your choice. **4** — communicate the framing, design choices, limitations and risks to a mixed technical and business audience.

1 · WORKING TITLE — Rubric 4 · ILO 4

- Name the system and what it does, not the topic. “X — forecasts Y for Z” beats “AI for Y”.
- Spellcheck it. It is the first thing I read, and it carries into your repository, your video and your pitch.
- If you cannot say it in eight words, section 2 is not decided yet.

2 · THE PROBLEM, AND WHY IT MATTERS — Rubric 1 (15%) · ILO 4

- One problem, one sentence. If your paragraph contains “and also”, you have two projects. Cut one.
- Never claim nobody has solved this. Spend thirty minutes searching, then name the closest existing tool and what it does *not* do. I will check this — and the gap you find becomes your section 5 argument.
- Quantify the pain — how often, how many people, how costly. “Worth solving” needs a number.
- State what is out of scope. I mark scope realism explicitly.

3 · WHO IT IS FOR, AND THE DOMAIN — Rubric 1 (15%) · ILO 4

- One primary user, named by role. Not “users”, not “everyone”.
- **Write a one-line persona** — a short, concrete sketch of one real person: who they are, what they are doing in the moment they reach for your system, and what they do and do not already know. “*Mei, a shift supervisor checking tomorrow’s roster on her phone at 11pm, who has never seen a confidence score*” tells you more about what to build than “operations

staff” ever will. Your persona decides your interface, your tone, and how much explanation each output needs.

- Say what that person does differently once your system works. If nothing changes, the problem is not real yet.
- Prefer a domain you already know. Domain knowledge is scarcer than Python and shows up in every later section.

4 · WHY AI — AND WHICH KIND — Rubric 2 (25%) · ILO 1, 3

- Decide. “Narrow machine learning could work, or maybe a foundation model” reads as undecided, and I will score it that way.
- Name your non-AI baseline — a rule, a lookup table, a keyword filter — and commit to measuring against it. Sometimes it wins. That is a finding, not a failure.
- **Match the type to the job, and climb the ladder only as far as you need.** Start with **prompting** a rented model — zero-shot, few-shot, chain-of-thought reasoning, structured output (Class 3). It is the cheapest thing that often works, and a well-engineered prompt with a decent evaluation set beats a badly-trained model. Move up only when you can say why: **retrieval-augmented generation (RAG)** when answers must be grounded in documents you hold (Class 2); **narrow machine learning** when you have labels and a bounded numeric or categorical prediction; **an agent** only when the task genuinely needs a multi-step loop with tools (Class 4). Every rung adds cost, latency and new failure modes — say what the extra rung bought you.
- **Show where a foundation model or large language model (LLM) fits — and fit it right.** Most of this course is about these models, so a project with none engages only a fraction of the syllabus. Give it a proper home: an explanation layer, a rented baseline you compare against, a judge inside your evaluation, help labelling data. Keep it out of arithmetic and safety-critical decision paths, where deterministic logic is checkable and a confident wrong answer is expensive. Right fit, not maximum usage — and if you decide against one, argue it explicitly rather than leaving it unmentioned.
- Say which parts stay rules. Most good answers are hybrids, and saying so earns marks rather than losing them.

5 · PROPOSED APPROACH — BUILD VS BUY — Rubric 2 (25%) · ILO 2, 3

- Work down the Class 2 stack layer by layer — interface and serving, orchestration, model, data and retrieval, evaluation and observability — and mark each one *own* or *rent*, with one reason.
- Name the actual thing you are renting: the library, the model, the service. Not “an API”.
- Rent commodity, own the logic specific to your problem. “Build what makes it unique” on its own is a slogan; prove it layer by layer.
- **Price it.** Cost per use — calls × tokens × price — plus time-to-deploy. Criterion 2 is scored on business axes as well as technical ones, and most statements do only the technical half. Two places in the course give you this: **Class 3’s third capsule** (Med-PaLM 2 → AI Wars) shows how the *technique* you choose drives the bill — chain-of-thought reasoning, self-consistency sampling and ensembling each multiply token spend for the same task — and **Class 5** turns that into a cost-to-serve calculator you can point at your own design.
- **Low-code is optional for this project — but if you tried it, show it.** Low-code means something you *configure* rather than program: a no-code model trainer (Google Teachable Machine, Google Vertex AI AutoML), a workflow builder (n8n, Zapier, Microsoft Power Automate), a hosted assistant builder, or a model playground where you set the prompt and parameters in a console. Calling a model from Python is **code**, not low-code. Many of you will reach for one of these before writing a line of code — that attempt is evidence, not a detour, so do not hide it. Write the one paragraph: what you tried, what it did well, and what pushed you to code (control, cost, latency, an integration it could not handle). “I tried X first, it broke on Y, so I built Z” is a stronger trade-off argument than a page of reasoning about tools you never opened. Going straight to code is fine and carries no penalty — but then say why you skipped the cheaper route.

6 · DATA — THIS IS WHERE PROJECTS DIE — Rubric 3 (35%) · ILO 2

- Name a specific dataset with a link and a row count, or the exact source you will collect from. “I will need labelled data” restates the question.
- Verify the dataset measures what you think it measures. Read the task definition in its paper — benchmarks are constantly used for jobs they were never built for, and a benchmark that sounds right can be measuring something else entirely.
- Permission and portability in one pass: check the licence; if you collect from people, consent and a purpose notice come *before* collection under Singapore’s Personal Data Protection Act; and if you cannot redistribute the data, plan now for a repository that runs without it — ship trained weights, a small sample, and a README that says exactly this.
- **If your data is synthetic** — data you generate rather than collect — that is a legitimate choice, and often the right one for a four-week build. Treat the generator as part of your system. **Commit the script or prompt that produced it** so a reader can regenerate it. **Fix the ground truth** — what the correct answer is for each case — in a file *before* you run anything, never after you have seen results. **Vary it deliberately** across length, tone, edge cases and awkward cases, or you will only ever prove that your system works on the narrow slice you imagined. And know its ceiling: data generated by the same

model you are testing inherits that model's blind spots — validate a handful of cases against something real, or state plainly what your synthetic set cannot tell you.

- Hunt for leakage — does the input already contain the answer? Strip it, and report the score before and after. That before-and-after is worth more to me than a high score.

7 · SUCCESS METRIC AND HOW YOU WILL EVALUATE IT — Rubric 3 (35%) · ILO 2, 3 · the evals thread

- One metric tied to value, with a target number and a baseline. Accuracy without the majority-class baseline — the score you get by always guessing the most common answer — says nothing.
- Check that your metric is internally consistent. Recall of 70% is a false-negative rate of 30% — you cannot target both.
- Make the evaluation set large enough that one flaky case does not move the headline, and do not grade your own homework: if you designed both the system and the test cases, hold out a set built by someone else or by a different model.
- **Give your system a way to say “I don’t know” — then measure it.** *Abstaining* means declining to answer rather than guessing; returning “not confident enough”, or handing the case to a person, when a confidence score falls below a threshold you choose. Report two numbers — **how often it abstains**, and **whether the cases it abstained on were the ones it would have got wrong**. A system that abstains on everything is safe and useless; one that never abstains is confident and dangerous. The threshold that balances those two is a design decision worth a paragraph.
- If a model judges your outputs, that judge is a component of your system, not a source of truth — measure its precision and recall against labels you assigned by hand.

8 · RISKS, LIMITATIONS AND RESPONSIBLE USE — Rubric 2, 3 and 4 · ILO 2, 3, 4 · Class 6

- Every risk needs its mitigation on the same line. A risk list without fixes is a description, not a plan.
- A disclaimer is not a mitigation. Confidence thresholds, abstention, human confirmation and rate limits are.
- Name the silent failure — the case where the system is confidently wrong and nobody notices — and say how you would detect it.
- State intended use and explicit non-use, especially for anything touching health, money, security, or inferences about a person. If the tool is dual-use, say who may run it and what you will not publish.
- Name frameworks precisely rather than gesturing at “regulation”: Singapore’s IMDA Model AI Governance Framework, the EU AI Act, or the OWASP Top 10 for Large Language Model Applications (2025) for AI security.

9 · SMALLEST FIRST VERSION — REQUIRED, NOT OPTIONAL — Rubric 3 (35%) · ILO 2

- One end-to-end path: one input → one model call → one output. Nothing alongside it — no second feature, no second model, no polish.
- A date is not a version. Describe the slice, not the deadline.
- Get it running early — that is a rule of thumb, not a requirement. A1 is due Sunday 30 August, so for most of you the realistic window for a first slice is the week after that. What matters is that the end-to-end path works well before the final week, because everything you add later is an extension of something that already runs.
- Write the one sentence that tells you it worked, and test it on real inputs. If that check fails, nothing built on top of it matters.

BEFORE YOU SUBMIT — FIVE CHECKS

1. Every section makes a decision, not a survey of options.
2. A named dataset, a named baseline, a named number.
3. First person throughout — this is individual work, and criterion 2 is scored on *your* reasoning.
4. Your repository will run on someone else’s machine.
5. Read it once for typos. Communication is 25% of the project mark.

Questions are welcome by email to the course account, or bring them to the class breakouts — whichever is easier for you. If you are wondering about something, several other people are too.